EXP1: To design and implement a Hidden Markov Model (HMM) for weather prediction using observable daily activities.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import accuracy_score, confusion_matrix
# ------------------------------
# Hidden States (Weather)
# ------------------------------
states = ["Sunny", "Rainy", "Cloudy"]
# Observable Activities
observations = ["Umbrella", "Sunglasses", "Jackets", "Normal"]
# Initial probabilities
start_prob = np.array([0.4, 0.3, 0.3])
# Transition Probability Matrix
transition_prob = np.array([
    [0.6, 0.2, 0.2],   # Sunny -> Sunny,Rainy,Cloudy
    [0.3, 0.5, 0.2],   # Rainy -> Sunny,Rainy,Cloudy
    [0.4, 0.3, 0.3]    # Cloudy -> Sunny,Rainy,Cloudy
])
# Emission Probability Matrix
# Umbrella Sunglasses Jackets Normal
emission_prob = np.array([
    [0.1, 0.6, 0.1, 0.2],   # Sunny
    [0.7, 0.05,0.2,0.05],   # Rainy
    [0.3,0.2,0.4,0.1]       # Cloudy
```

```
])
# Example observation sequence:
# Umbrella, Jackets, Sunglasses
obs_sequence = [0,2,1]
# ----------------------------------
# Forward Algorithm
# ----------------------------------
def forward_algorithm(obs_seq):
    N = len(states)
    T = len(obs_seq)
    alpha=np.zeros((N,T))
    # Initialization
    for i in range(N):
        alpha[i,0]=start_prob[i]*emission_prob[i,obs_seq[0]]
    # Recursion
    for t in range(1,T):
        for j in range(N):
            alpha[j,t]=np.sum(
                alpha[:,t-1]*transition_prob[:,j]
            )*emission_prob[j,obs_seq[t]]
    probability=np.sum(alpha[:,T-1])
    return alpha,probability
alpha_matrix,sequence_probability=forward_algorithm(obs_sequence)
print("Forward Probability Matrix:\n")
print(alpha_matrix)
print("\nObservation Sequence Probability:")
```

```
print(sequence_probability)
# ----------------------------------
# Viterbi Algorithm
# ----------------------------------
def viterbi(obs_seq):
    n_states=len(states)
    T=len(obs_seq)
    dp=np.zeros((n_states,T))
    backpointer=np.zeros((n_states,T),dtype=int)
    # Initialization
    dp[:,0]=start_prob*emission_prob[:,obs_seq[0]]
    # Recursion
    for t in range(1,T):
        for s in range(n_states):
            prob=dp[:,t-1]*transition_prob[:,s]
            dp[s,t]=np.max(prob)*emission_prob[s,obs_seq[t]]
            backpointer[s,t]=np.argmax(prob)
    # Backtracking
    best_path=[np.argmax(dp[:,T-1])]
    for t in range(T-1,0,-1):
        best_path.insert(
            0,
            backpointer[best_path[0],t]
        )
    return [states[i] for i in best_path]
predicted_states=viterbi(obs_sequence)
```

```
print("\nPredicted Hidden States:")
print(predicted_states)
# ---------------------------------
# Evaluation
# ---------------------------------
# Ground truth for experiment
actual_states=["Rainy","Cloudy","Sunny"]
state_to_index={
    state:idx for idx,state in enumerate(states)
}
actual_numeric=[
    state_to_index[s]
    for s in actual_states
]
predicted_numeric=[
    state_to_index[s]
    for s in predicted_states
]
accuracy=accuracy_score(
    actual_numeric,
    predicted_numeric
)
print("\nAccuracy:",accuracy)
# ---------------------------------
# Confusion Matrix
# ---------------------------------
```

```
cm=confusion_matrix(
    actual_numeric,
    predicted_numeric
)
plt.figure(figsize=(6,4))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=states,
    yticklabels=states
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix for Weather Prediction")
plt.show()
# ----------------------------------
# Time Series Visualization
# ----------------------------------
plt.figure(figsize=(8,4))
plt.plot(
    actual_numeric,
    marker='o',
    label='Actual'
)
```

```
plt.plot(
    predicted_numeric,
    marker='s',
    label='Predicted'
)
plt.yticks(
    range(len(states)),
    states
)
plt.xlabel("Time Step")
plt.ylabel("Weather State")
plt.title(
    "Actual vs Predicted Weather States"
)
plt.grid()
plt.legend()
plt.show()
```

EXP2: To design and implement a Bayesian Network for predicting whether a student will pass a final exam based on study habits, class attendance, and internal assessment performance.

```
from sklearn.naive_bayes import GaussianNB
import pandas as pd
# Dataset
data = pd.DataFrame({
'StudyHours':[0,1,1,0,1,1,0,0],
'Attendance':[0,1,0,1,1,1,0,1],
```

```
'InternalMarks':[0,1,1,0,1,0,0,1],
# 0 = Fail
# 1 = Pass
'Pass':[0,1,1,0,1,1,0,1]
})
# Inputs
X=data[['StudyHours',
        'Attendance',
        'InternalMarks']]
# Output
y=data['Pass']
# Train model
model=GaussianNB()
model.fit(X,y)
# Prediction
# High Study=1
# Regular Attendance=1
# High Internal Marks=1
prediction=model.predict([[1,1,0]])
if prediction[0]==1:
    print("Prediction: Student will PASS")
else:
    print("Prediction: Student will FAIL")
```

EXP3: To design and implement a Gaussian Mixture Model (GMM) for outcome prediction on a real-world dataset.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score
# ----------------------------------
# 1. Generate Sample Data
# ----------------------------------
np.random.seed(42)
X1 = np.random.multivariate_normal(
    mean=[-2,-2],
    cov=[[0.5,0.1],[0.1,0.5]],
    size=150
)
X2 = np.random.multivariate_normal(
    mean=[2,-2],
    cov=[[0.6,-0.2],[-0.2,0.6]],
    size=150
)
X3 = np.random.multivariate_normal(
    mean=[2,2],
    cov=[[0.4,0.0],[0.0,0.4]],
    size=150
)
X=np.vstack((X1,X2,X3))
# true labels only for evaluation
```

```
true_labels=np.array(
    [0]*150+[1]*150+[2]*150
)
# ----------------------------------
# 2. Fit Gaussian Mixture Model
# ----------------------------------
gmm=GaussianMixture(
    n_components=3,
    covariance_type='full',
    random_state=42
)
gmm.fit(X)
labels=gmm.predict(X)
# ----------------------------------
# 3. Visualize Cluster Results
# ----------------------------------
plt.figure(figsize=(8,6))
sns.scatterplot(
    x=X[:,0],
    y=X[:,1],
    hue=labels,
    palette='viridis'
)
plt.scatter(
    gmm.means_[:,0],
    gmm.means_[:,1],
```

```
    c='red',
    marker='X',
    s=250,
    label='Cluster Means'
)
plt.title("Gaussian Mixture Model Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()
# ----------------------------------
# 4. Model Evaluation
# ----------------------------------
ari=adjusted_rand_score(
    true_labels,
    labels
)
print("Adjusted Rand Index:",ari)
# ----------------------------------
# 5. Probability Predictions
# ----------------------------------
posterior_prob=gmm.predict_proba(X[:5])
print("\nPosterior Probabilities:")
for i,p in enumerate(posterior_prob):
    print(f"Sample {i+1}: {p}")
# ----------------------------------
```

```
# 6. Cluster Means
# ----------------------------------
print("\nCluster Means:")
print(gmm.means_)
# ----------------------------------
# 7. Covariance Matrices
# ----------------------------------
print("\nCluster Covariances:")
for i,cov in enumerate(gmm.covariances_):
    print(f"\nGaussian Component {i+1}")
    print(cov)
```

EXP4: To build and train a Generative Multi-Layer Network Model that maps a low-dimensional latent space to a target data distribution. This experiment will demonstrate how a feed forward neural network (a multilayer perceptron) can be used as a generative model for outcome generation.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Dense
import matplotlib.pyplot as plt
# --------------------------------
# 1. Generate Training Data
# --------------------------------
np.random.seed(42)
num_samples = 5000
latent_dim = 5
```

```
# latent noise
z = np.random.normal(
    0,
    1,
    (num_samples, latent_dim)
)
# target outputs
Y = np.zeros((num_samples,2))
Y[:,0] = np.sin(z[:,0])
Y[:,1] = np.cos(z[:,1])
# ---------------------------------
# 2. Build Model
# ---------------------------------
model = Sequential([
    Input(shape=(latent_dim,)),
    Dense(
        64,
        activation='relu'
    ),
    Dense(
        32,
        activation='relu'
    ),
    Dense(2)
])
# ---------------------------------
```

```
# 3. Compile
# ---------------------------------
model.compile(
    optimizer='adam',
    loss='mse'
)
# ---------------------------------
# 4. Train
# ---------------------------------
history=model.fit(
    z,
    Y,
    epochs=100,
    batch_size=32,
    verbose=1
)
# ---------------------------------
# 5. Generate New Samples
# ---------------------------------
z_new=np.random.normal(
    0,
    1,
    (1000,latent_dim)
)
generated=model.predict(z_new)
# ---------------------------------
```

```
# 6. Plot Results
# --------------------------------
plt.figure(figsize=(8,6))
plt.scatter(
    generated[:,0],
    generated[:,1],
    alpha=0.5
)
plt.title(
    "Generated Outputs from Generative Model"
)
plt.xlabel(
    "Output Dimension 1 (sin)"
)
plt.ylabel(
    "Output Dimension 2 (cos)"
)
plt.grid()
plt.show()
```

EXP5: To build and train a Deep Convolution Generative Adversarial Network (DCGAN) for generating realistic images from a given image dataset.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets import mnist
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Dense,
    Reshape,
    Flatten,
    Conv2D,
    Conv2DTranspose,
    LeakyReLU,
    Dropout
)
from tensorflow.keras.optimizers import Adam
# Load dataset
(X_train, _), (_, _) = mnist.load_data()
# Normalize images
X_train = (X_train.astype('float32') - 127.5) / 127.5
X_train = np.expand_dims(X_train, axis=-1)
# Generator
generator = Sequential([
    Dense(7 * 7 * 128, input_dim=100),
    LeakyReLU(0.2),
    Reshape((7, 7, 128)),
    Conv2DTranspose(
        64,
        kernel_size=4,
        strides=2,
        padding='same'
```

```
    ),
    LeakyReLU(0.2),
    Conv2DTranspose(
        1,
        kernel_size=4,
        strides=2,
        padding='same',
        activation='tanh'
    )
])
# Discriminator
discriminator = Sequential([
    Conv2D(
        64,
        kernel_size=4,
        strides=2,
        padding='same',
        input_shape=(28, 28, 1)
    ),
    LeakyReLU(0.2),
    Dropout(0.3),
    Flatten(),
    Dense(1, activation='sigmoid')
])
# Compile discriminator
discriminator.compile(
```

```
    optimizer=Adam(0.0002),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
# Build GAN
discriminator.trainable = False
gan = Sequential([
    generator,
    discriminator
])
# Compile GAN
gan.compile(
    optimizer=Adam(0.0002),
    loss='binary_crossentropy'
)
# Training
epochs = 200
batch_size = 64
for epoch in range(epochs):
    # Real images
    idx = np.random.randint(
        0,
        X_train.shape[0],
        batch_size
    )
    real_imgs = X_train[idx]
```

```
# Fake images
noise = np.random.normal(
    0,
    1,
    (batch_size, 100)
)
fake_imgs = generator.predict(
    noise,
    verbose=0
)
# Train discriminator
d_loss_real = discriminator.train_on_batch(
    real_imgs,
    np.ones((batch_size, 1))
)
d_loss_fake = discriminator.train_on_batch(
    fake_imgs,
    np.zeros((batch_size, 1))
)
# Train generator
noise = np.random.normal(
    0,
    1,
    (batch_size, 100)
)
g_loss = gan.train_on_batch(
```

```
        noise,
        np.ones((batch_size, 1))
    )
    # Print progress
    if epoch % 50 == 0:
        print(
            f"Epoch {epoch} | "
            f"D Loss: {float(d_loss_real[0]):.4f} | "
            f"G Loss: {float(g_loss):.4f}"
        )
# Generate images
noise = np.random.normal(
    0,
    1,
    (5, 100)
)
generated_imgs = generator.predict(noise)
# Plot generated images
plt.figure(figsize=(10, 2))
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(
        generated_imgs[i, :, :, 0],
        cmap='gray'
    )
    plt.axis('off')
```

```
plt.suptitle("Generated MNIST Images")
plt.show()
```

EXP6: To explore the working of a pre-trained model (VGG16) for outcome generation and to use transfer learning to classify images from a custom dataset.

```
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Flatten,
    Dense,
    Dropout
)
# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
# Resize images for VGG16
x_train = tf.image.resize(x_train, (48, 48))
x_test = tf.image.resize(x_test, (48, 48))
# Normalize images
x_train = x_train / 255.0
x_test = x_test / 255.0
# Load VGG16
base_model = VGG16(
    weights='imagenet',
    include_top=False,
```

```
    input_shape=(48, 48, 3)
)
# Freeze layers
for layer in base_model.layers:
    layer.trainable = False
# Create model
model = Sequential([
    base_model,
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])
# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
# Train model
history = model.fit(
    x_train,
    y_train,
    epochs=2,
    validation_split=0.2,
    batch_size=64
```

```
)
# Evaluate model
loss, accuracy = model.evaluate(
    x_test,
    y_test
)
print("\nTest Accuracy:", accuracy)
# Prediction
prediction = model.predict(x_test[:1])
print(
    "Predicted Class:",
    prediction.argmax()
)
# Display image
plt.imshow(x_test[0])
plt.title("Sample Test Image")
plt.axis('off')
plt.show()
```

EXP7: To implement and analyze the working of the Local Interpretable Model-Agnostic Explanations (LIME) framework to interpret model predictions in a supervised classification problem.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
```

```
from lime import lime_tabular
# Load Iris Dataset
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
class_names = iris.target_names
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
# Train Random Forest Model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
model.fit(X_train, y_train)
# Model Accuracy
accuracy = model.score(X_test, y_test)
print("Model Accuracy:", accuracy)
# Create LIME Explainer
explainer = lime_tabular.LimeTabularExplainer(
    training_data=X_train,
    feature_names=feature_names,
```

```
    class_names=class_names,
    mode='classification'
)
# Select Sample
index = 0
sample = X_test[index]
# Explain Prediction
explanation = explainer.explain_instance(
    data_row=sample,
    predict_fn=model.predict_proba,
    num_features=4
)
# Print Feature Contributions
print("\nLIME Explanation:")
for feature, weight in explanation.as_list():
    print(f"{feature}: {weight:.4f}")
# Predicted Class
predicted_class = model.predict([sample])[0]
print("\nPredicted Class:",
      class_names[predicted_class])
# Plot Explanation
fig = explanation.as_pyplot_figure()
plt.title("LIME Explanation")
plt.show()
```